

第10章 运算符重载

10.1 重载运算符

对于内置类型，可以直接进行加减乘除等运算，这些运算的语义是预定义且明确的。而对于自定义类型，若希望其也能参与类似的运算，则必须自定义这些运算的具体含义。

通常的做法是通过成员函数或非成员函数的形式，重新定义这些运算符的语义行为。例如，下面的 `TVector` 类：

```
1  class TVector {
2  public:
3      TVector(int x = 0,int y = 0):mX(x),mY(y) {}
4      int getX( ) const { return mX; }
5      int getY( ) const { return mY; }
6      ...
7  private:
8      int mX;
9      int mY;
10 };
```

要实现两个 `TVector` 对象的相加，可以通过为 `TVector` 类添加一个成员函数 `addVector`，或者定义一个自由函数 `addVector`。此时，主函数可以采用以下形式：

```
1  int main( ){
2      TVector v1;
3      TVector v2;
4      TVector result1 = v1.addVector( v2 ); //成员函数形式
5      TVector result2 = addVector( v1, v2 ); //自由函数形式
6  }
```

这里的问题是，与直观且语义清晰的 `v1 + v2` 相比，`addVector` 这样的函数名显得不够直观。由于 '+' 不能作为函数名的标识符，C++ 引入了 `operator` 关键字，允许用户通过定义 `operator+` 形式的函数来为自定义类型指定加法运算的语义。同理也可以自定义其它运算符的语义，这就是运算符重载。

C++中的运算符可分为一元、二元和多元运算符。这里的“元”指的是运算符关联的操作数数量：一元运算符作用于一个操作数，二元运算符作用于两个操作数，而多元运算符(如条件运算符)则作用于多个操作数，C++中，唯一的多元运算符是三元条件运算符(?:)。运算符的元数与其字符串长度、

位置或表达形式无关。例如，前置递增运算符(++)尽管由两个字符组成，仍是一元运算符；赋值运算符(=)虽然只有一个字符，却是二元运算符。加法运算符通常以中缀形式出现，但即使使用前缀或后缀表示法，它仍然是二元运算符。对于取分量运算符([])，常见的语法格式是 a[index]，运算符的各部分完全分开书写了，但仍被视为单个运算符。

要重载运算符，需要使用 operator 关键字，其语法格式有有多种，本章只涉及下表中的第一类。

表 10-1 operator 关键字的多种语法格式

语法格式	说明
operator 运算符	最常见的运算符重载
operator 类型名	类型转换函数
operator new operator new [] operator delete operator delete []	动态内存分配和释放
operator "" 后缀标识符	C++11 新增，用户自定义文字量

此外，重载运算符可以采用成员函数或非成员函数的形式。成员函数通常是类的实例方法；而非成员函数也称为自由函数，这里的“自由”意味着调用时无需像实例方法那样限定作用域(Scope)。

常见的非成员函数包括全局自由函数、命名空间中的自由函数或类方法。例如：

```
1 //全局自由函数
2 B operator+( const B& b1, const B& b2 )
3     { /*略*/ }
4 namespace NS {
5     //名字空间中的自由函数
6     C operator+(const C& c1, const C& c2 )
7         { /*略*/ }
8
9     class D {
10    public:
11        //类方法
12        static D operator+( const D& d1, const D& d2)
13            { /*略*/}
14    };
15
16    class E {
17    public:
18        //实例方法
19        E operator+(const E& e ) const
20            { /*略*/ }
21    };
```

22 }

重载运算符时需遵循以下限制。

- (1) 只能重载一元或二元运算符，不允许重载三元条件运算符。
- (2) 操作数中至少有一个必须是用户自定义类型，即不能改变内置类型的运算语义。
- (3) 无法改变运算符的结合律、优先级等固有特性。
- (4) 不允许定义新的运算符，例如 `**`、`<>`、`&|` 等。
- (5) 下列运算符不可重载：成员访问运算符`(.)`、成员指针访问运算符`(.*)`、作用域解析运算符`::`、条件运算符`(?:)`、`sizeof`、`typeid`、`static_cast`、`dynamic_cast`、`const_cast`、`reinterpret_cast` 等。

下表 10-2 可重载的运算符列出了可重载的运算符。

表 10-2 可重载的运算符

可重载的运算符	说明
<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> , <code>^=</code> , <code>&=</code> , <code> =</code> , <code>>>=</code> , <code><<=</code>	赋值运算符
<code>++</code> , <code>--</code>	递增/递减运算符
<code>+(正号)</code> , <code>-(负号)</code> , <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>&</code> , <code> </code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code>	算术运算符
<code>!</code> , <code>&&</code> , <code> </code>	逻辑运算符
<code>==</code> , <code><=></code> , <code>!=</code> , <code>></code> , <code>>=</code> , <code><</code> , <code><=</code> , (C++20 用 <code><=></code> 取代后边 5 个)	比较运算符
<code>[]</code> , <code>->*</code> , <code>-></code>	成员访问运算符
<code>,</code> , <code>()</code>	逗号运算符和函数调用运算符
<code>operator new</code> , <code>operator new[]</code> , <code>operator delete</code> , <code>operator delete[]</code>	动态内存分配和释放

10.2 重载二元运算符

假设， 有自定义类 TVector 如下：

```
1 class TVector {
2 public:
3     A(int x = 0,int y = 0):mX(x),mY(y) { }
4     ...
5 private:
6     int mX;
7     int mY;
8 };
```

10.2.1 运算符重载

下面我们以 TVector 对象的加法运算为例，说明二元运算符的重载。

通常，重载运算符有两种形式：成员函数和非成员函数。对于 TVector 类：

- 如果采用非成员函数形式，则定义为：TVector operator+(const TVector& v1, const TVector& v2)。此时，表达式 v1 + v2 会被编译为 operator+(v1, v2)。
- 如果采用成员函数形式，则定义为：TVector TVector::operator+(const TVector& rhs)。此时，表达式 v1 + v2 会被编译为 v1.operator+(v2)。

关于哪种形式更合适，我们将在后面给出建议。但无论选择哪种形式，重载运算符都不能改变运算符的优先级或结合性等固有特性。例如，如果使用成员函数形式，表达式 a + b + c * d 会编译为 a.operator+(b).operator+(c.operator*(d))。同样地，表达式 a = b = c 会编译为 a.operator=(b.operator=(c))。

定义重载函数时，应明确指定函数的返回值类型和参数类型。建议如下：

1. 按值返回，还是按引用返回：可以对照内置类型运算的语义，例如：

!a <==>	bool T::operator!() const	//成员函数形式
a*b <==>	T operator*(const T& t1, const T& t2)	//自由函数形式
a=b <==>	T& T::operator=(const T& t)	//成员函数形式

2. 返回值是否带 const 修饰：引用返回时，参考内置类型的语义；值返回时，一般不带 const，例如：

a*=b <==>	T& T::operator*=(const T& t)	//成员函数形式
a*b <==>	T operator*(const T& t1, const T& t2)	//自由函数形式

3. 函数参数是否带 const 修饰：按照运算符语义，若不改变实参，就带 const，否则，不带 const。
4. 成员函数是否带 const 修饰：同样按照运算符语义，若不改变对象，就带 const，否则，不带 const。例如：

!a <==>	bool T::operator!() const	//成员函数形式
a*=b <==>	T& T::operator*=(const T& t)	//成员函数形式

10.2.2 重载的实现

采用成员函数或非成员函数的形式实现运算符重载，其语义都不复杂。

例如，以自由函数的形式实现时，代码可能如下所示：

1	class TVector {
2	friend TVector operator+(const TVector &, const TVector &);
3	public:

```

4     TVector(int x = 0,int y = 0):mX(x),mY(y) { }
5 private:
6     int mX;
7     int mY;
8 };
9 //自由函数形式重载 operator+
10 TVector operator+(const TVector & lhs,const TVector & rhs) {
11     TVector result;
12     result.mX = lhs.mX+rhs.mX;
13     result.mY = lhs.mY+rhs.mY;
14     return result;
15 }

```

以成员函数的形式实现时，代码可能如下所示：

```

1 class TVector {
2 public:
3     A(int x = 0,int y = 0):mX(x),mY(y) { }
4     TVector operator+(const TVector & rhs) {
5         TVector result;
6         result.mX = mX + rhs.mX;
7         result.mY = mY + rhs.mY;
8         return result;
9     }
10 private:
11     int mX;
12     int mY;
13 };

```

10.2.3 返回值优化(RVO)

在上述实现中，首先创建了一个局部对象，然后对其进行修改，随后通过多次拷贝构造返回新对象，并最终释放局部对象。这种做法效率较低，因为多次拷贝构造并没有带来实质性的价值，而编译器也难以理解具体的程序语义，从而无法对此过程进行优化。为了使编译器能够更好地优化代码，应该在语法层面就为其提供充分的优化信息。因此，可以对实现进行如下改进(成员函数形式同样适用)：

```

1 class TVector {
2     friend TVector operator+(const TVector & lhs,const TVector & rhs);
3 public:
4     TVector(int x = 0,int y = 0):mX(x),mY(y) { }
5 private:
6     int mX;

```

```

7     int mY;
8 };
9 //自由函数形式重载 operator+
10 TVector operator+(const TVector & lhs,const TVector & rhs) {
11     return TVector(lhs.mX+rhs.mX,lhs.mY+rhs.mY);
12 }

```

其中，`return` 语句返回的是一个匿名的临时对象，该对象的生命周期仅持续到当前语句结束。因此，编译器可以轻易判断出需要返回一个新的对象，并且意识到这个新对象没有必要在栈上创建，因为即使创建了也会迅速被销毁。所以，编译器可以将这段代码优化为：在函数调用处直接生成返回的新对象。这就是返回值优化(RVO)。

10.2.4 成对重载

运算符中有很多是成对出现的，例如 `+` 和 `+=`。通常情况下，如果允许两个 `TVector` 对象之间的 `+` 操作，也应当支持 `+=` 操作。因此，在重载 `+` 运算符时，最好同时重载对应的 `+=` 运算符。同样的原则适用于其他需要成对重载的运算符，比如 `==` 和 `!=`、`&` 和 `|` 等。以下是重载 `+` 和 `+=` 运算符的示例代码：

```

1  class TVector {
2  public:
3      TVector(int x = 0,int y = 0):mX(x),mY(y) { }
4      TVector & operator+=(const TVector& rhs) {
5          mX += rhs.mX;
6          mY += rhs.mY;
7          return *this;
8      }
9  private:
10     int mX;
11     int mY;
12 };
13 //自由函数形式重载 operator+
14 TVector operator+(const TVector & lhs,const TVector & rhs) {
15     return TVector(lhs) += rhs;
16 }

```

从上述代码可以看出成对重载的优势：

- (1) 支持用户不同的使用习惯，既可以使用 `v1 + v2` 的形式，也可以使用 `v1 += v2` 的形式。
- (2) 在 `TVector` 类中无需声明友元函数 `TVector operator+(const TVector &lhs, const TVector &rhs)`，因为 `operator+` 的实现不再需要直接访问 `TVector` 的私有成员，而是通过调用公

有成员函数 `operator+=` 来完成操作。

- (3) 当需要调整加法操作的语义时，只需修改 `operator+=` 的实现，而无需改动 `operator+`。这不仅简化了维护工作，还确保了 `+` 和 `+=` 运算符在加法语义上的一致性。

10.3 重载一元运算符

假设 `TVector` 的递增和递减操作分别表示其分量 `mX` 的增加和减少，那么重载递增和递减运算符的代码可如下实现：

```

1  class TVector {
2  public:
3      TVector( int x = 0, int y = 0 ) :mX( x ), mY( y ) { }
4      bool operator!( ) const { return ( mX != 0 && mY != 0 ); }
5      TVector & operator++( )          //前置++
6          { ++mX; return *this; }
7      TVector operator++( int )        //后置++
8          { TVector temp( *this ); ++( *this ); return temp; }
9      TVector & operator--( )          //前置--
10         { --mX; return *this; }
11     TVector operator--( int )        //后置--
12         { TVector temp( *this ); --( *this ); return temp; }
13 private:
14     int mX;
15     int mY;
16 };

```

需要注意前置++和后置++在语义上的区别。为了区分前置和后置操作符的重载函数，通常需要为它们指定不同的参数列表。对于后置操作符函数，一般会添加一个未使用的整型参数，尽管这个参数在函数实现中并未真正使用，但它确实存在并且会被压入栈中。例如，`++v` 会被编译为 `v.operator++()`，而 `v++` 则通常会被编译为 `v.operator++(0)` 或 `v.operator++(1)`，这里的参数 `0` 或 `1` 仅用于区分重载函数。此外，还应注意前置和后置操作在执行效率上存在差异。

10.4 其它的运算符重载

10.4.1 重载 `[]` 运算符

`[]` 运算符是一个分量访问运算符。要重载该运算符，可如下实现：

```

1  #include <iostream>
2  using namespace std;

```

```

3  class IntArray {
4  public:
5      int & operator[]( int index ) {
6          if ( index >= 0 && index < SIZE )
7              return mData[ index ];
8          throw "下标越界";
9      }
10     int operator[]( int index ) const {
11         if ( index >= 0 && index < SIZE ) {
12             return mData[ index ];
13         }
14         throw "下标越界";
15     }
16 public:
17     static const int SIZE = 3;
18 private:
19     int mData[ SIZE ] {};
20 };
21 int main( ) {
22     IntArray vArr;
23     for ( int i = 0; i < IntArray::SIZE; ++i ) {
24         vArr[ i ] = i + 1;          //调用 int & IntArray::operator[]( int )
25     }
26     for ( int i = 0; i < IntArray::SIZE; ++i ) {
27         cout << vArr[ i ] << " "; //调用 int & IntArray::operator[]( int )
28     }
29     cout << endl;
30     const IntArray cArr( vArr );
31     for ( int i = 0; i < IntArray::SIZE; ++i ) {
32         cout << cArr[ i ] << " "; //调用 int IntArray::operator[]( int ) const
33     }
34     cout << endl;
35 }

```

在重载[]运算符时, 建议同时提供带 `const` 修饰和不带 `const` 修饰的版本, 以确保既能操作普通对象也能操作常对象。例子中的各分量是整数, 如果成员是对象, 则可以这样定义重载函数:

```

1  class ObjectArray {
2  public:
3      Object & operator[](int index) { return mObjects[index]; }
4      const Object & operator[](int index) const { return mObjects[index]; }
5  private:
6      Object mObjects[3];
7  };

```


10.4.2 重载->运算符

考虑下面的代码：

```

1  class ImpA {
2  public:
3      void f1( ) { /* 略 */ }
4      void f2( ) { /* 略 */ }
5      /* 其它略 */
6  private:
7      //数据成员,略
8  };
9  class A {
10 public:
11     void f1( ) { mpImpA->f1( ); }
12     void f2( ) { mpImpA->f2( ); }
13     /* 其它略 */
14 private:
15     ImpA * mpImpA;
16 };

```

这里，类 A 的功能实际上是在 ImpA 类中实现的。类 A 中的各个函数只是简单地转发请求。这种设计体现了接口与实现分离的思想：用户只需与类 A 交互，无需关心具体的数据组织和实现细节。特别是当修改 ImpA 类中的 f1、f2 等函数的实现时，类 A 和用户程序都不需要更改。实际上，标准库中 string 类的设计和实现也采用了这种方式。

然而，如果要在类 ImpA 中添加一个新的成员函数 f3()，就需要同步修改类 A 的代码。而通过重载运算符->，可以避免修改类 A 的代码。例如：

```

1  class ImpA {
2  public:
3      void f1( ) { /* 略 */ }
4      void f2( ) { /* 略 */ }
5      void f3( ) { /* 略 */ }
6  private:
7      //数据成员,略
8  };
9  class A {
10 public:
11     A( ) : mpImpA(new ImpA) { }
12     ~A( ) { delete mpImpA;}
13     ImpA * operator->( )          { return mpImpA;}    //重载 operator->
14     const ImpA * operator->( ) const { return mpImpA;}  //重载 operator->

```

```

15 private:
16     ImpA * mpImpA;
17 };
18 int main( ) {
19     A aA;
20     aA->f1( );
21     aA->f2( );
22     aA->f3( );
23 }

```

类 A 中不再显式定义公共成员函数 f1、f2 和 f3，但仍然可以通过 aA->f1()、aA->f2() 和 aA->f3() 的形式访问 ImpA 中的相应函数。实际上，由于重载了运算符->，当调用 aA->f1()时，编译器会将其转换为(aA.operator->())->f1()。这表明，通过重载运算符->，可以实现函数调用的转发或委托机制。

根据 C++标准，当在类型 T 中重载 operator->时，必须满足以下两种情况之一：

- 类 T 中的 operator->函数返回一个自定义类型的指针。
- 类 T 中的 operator->函数返回一个自定义类型的对象或引用，并且该自定义类型也重载了 operator->。

例如：

```

1  #include <iostream>
2  using namespace std;
3  struct A {
4      int sum( ) const { return mX + mY; }
5      int mX = 10;
6      int mY = 20;
7  };
8  struct B {
9      A * operator->( )          { return mpA; } //返回 A 的指针
10     const A * operator->( ) const { return mpA; } //返回 A 的指针
11     int z = 30;
12     A * mpA;
13 };
14 struct C {
15     B & operator->( )          { return *mpB; } //返回 B 的引用,且 B 中重载了 operator->
16     const B & operator->( ) const { return *mpB; } //返回 B 的引用,且 B 中重载了 operator->
17     int k = 40;
18     B * mpB;
19 };
20 int main( ) {

```

```

21     A a { 3, 5 };
22     B b { 5, &a };
23     C c { 6, &b };
24
25     cout << a.sum( ) << endl;
26     cout << b->sum( ) << endl; // ( b.operator->( ) ) -> sum( )
27     cout << c->sum( ) << endl; // ( c.operator->( ).operator->( ) ) -> sum( )
28 }

```

10.4.3 重载()运算符

()运算符也称函数调用运算符。例如：

```

1  #include <iostream>
2  using namespace std;
3  class T {
4  public:
5      T( int a = 3, int b = 5 ) : mA( a ), mB( b ) { }
6      int operator( ) ( int factor ) const { return ( mA + mB ) * factor; }
7      double operator( )( double factor ) const { return factor * mA / mB; }
8  private:
9      int mA;
10     int mB;
11 };
12 int main( ) {
13     T aT;
14     cout << aT( 2 ) << endl; //输出 16
15     cout << aT( 3.0f ) << endl; //输出 1.8
16 }

```

通过类 T 中重载的两个 operator()函数，相当于为对象提供了类似函数调用的行为，使对象可以像函数一样被调用，这被称为仿函数(Functor)。那么，仿函数有什么用途呢？我们知道，函数参数不仅可以是变量、对象或指针，还可以是另一个函数。传递函数有以下几种常见方式：

- (1) 使用 C 语言支持的函数指针，将函数指针作为实参传递。
- (2) 利用继承和虚函数机制，传递父类对象的引用或指针（详见虚机制）。
- (3) 使用这里提到的仿函数，即把重载了 operator()的类对象作为函数参数。
- (4) 使用 lambda 表达式。
- (5) 使用 std::function 模板

重载运算符()的主要目的是应用于第三种方式。从示例代码中可以看出，当类 T 定义完成后，其仿函数的功能和语义也就固定了。如果需要改变仿函数的语义，就必须修改类 T 的定义，这不符

合复用的原则。实际上，重载运算符()的主要价值在于与模板结合，在模板库中有着广泛的应用。

以下是一个示例：

```

1  #include <iostream>
2  using namespace std;
3  class T1 {
4  public:
5      int operator( )( ) const { return 1; }
6  };
7  class T2 {
8  public:
9      int operator( )( ) const { return 2; }
10 };
11 template <typename T>
12 class MyClass {
13 public:
14     void func( ) { cout << T( )( ) << endl; }
15     void func( const T & aT ) { cout << aT( ) << endl; }
16 };
17 template <typename T>
18 void func( ) { cout << T( )( ) << endl; }
19 template <typename T>
20 void func( const T & aT ) { cout << aT( ) << endl; }
21 int main( ) {
22     T1 t1;
23     MyClass<T1> obj1;
24     obj1.func( );           //输出 1
25     obj1.func( t1 );       //输出 1
26     T2 t2;
27     MyClass<T2> obj2;
28     obj2.func( );           //输出 2
29     obj2.func( t2 );       //输出 2
30     func<T1>( );           //输出 1
31     func<T1>( T1( ) );     //输出 1
32     func<T2>( );           //输出 2
33     func<T2>( T2( ) );     //输出 2
34 }

```

以 MyClass 类模板为例，在其两个 func 函数的实现中需要用到某些特定功能。这些功能的具体实现被定义在 T1 或 T2 类中。如果需要更改所用的功能，可以创建一个新的 T3 类，并在其中重载 () 运算符，然后使用 T3 类型作为模板参数实例化 MyClass 类模板即可。整个过程中无需修改 MyClass 类模板的定义。

10.4.4 重载<<和>>运算符

我们经常使用 `cout << value1 << value2;` 的形式将内容输出到屏幕, 以及使用 `cin >> var1 >> var2;` 的形式从键盘获取输入。如果 `value1`、`value2`、`var1` 和 `var2` 是内置类型, 可以直接使用上述语法; 然而, 对于自定义类型, 则需要重载 `<<` 和 `>>` 运算符以定义其输入和输出行为。

`<<`(输出)和 `>>`(输入)运算符均为二元运算符, 其中左操作数为流对象, 而右操作数才是要处理的实际数据或对象。对于 `<<` 运算符而言, 左操作数是输出流对象, 即 `ostream` 类或其后裔类的对象, 它可以是文件流、字节流、字符流等。`cout` 仅是预定义的一个输出流对象, 代表控制台输出, 通常就是屏幕。对于 `>>` 运算符, 左操作数是输入流对象, 即 `istream` 类或其后裔类的对象, 它可以是文件流、字节流、字符流等。`cin` 仅是预定义的一个输入流对象, 代表控制台输入, 通常就是键盘。

在自定义类型 `T` 中重载 `<<` 和 `>>` 运算符时, 由于左操作数必须是流对象而非 `T` 类型的对象, 因此只能以非成员函数的形式定义重载函数。例如:

```

1  #include <iostream>
2  using namespace std;
3  class TVector {
4  public:
5      TVector(int x = 0,int y = 0):mX(x),mY(y) { }
6  public:
7      int getX( ) const { return mX;}
8      void setX(int x) { mX = x;}
9      int getY( ) const { return mY;}
10     void setY(int y) { mY = y; }
11 private:
12     int mX;
13     int mY;
14 };
15 ostream & operator<<(ostream & out, const TVector & vct ) {
16     out << "TVector: [ " << vct.getX( ) << " , " << vct.getY( ) << " ]" << endl;
17     return out;
18 }
19 istream & operator>>(istream & in, TVector & vct ) {
20     int x = 0, y = 0;
21     in >> x >> y;
22     vct.setX( x );
23     vct.setY( y );
24     return in;
25 }
26 int main( ) {

```

```

27     TVector v1( 3, 5 );
28     TVector v2( 9, 12 );
29     cout << v1 << v2; //输出 TVector: [ 3 , 5 ] 和 TVector: [ 9 , 12 ]两行
30     cin >> v1 >> v2; //若键盘输入 10 20 30 40
31     cout << v1 << v2; //则输出 TVector: [ 10 , 20 ] 和 TVector: [ 30 , 40 ]两行
32 }

```

10.4.5 重载文字量运算符

在定义文字量时，有时需要同时指定单位或特殊含义，这就需要自定义文字量了。例如：

```

1  #include <string>
2  #include <chrono>
3  int main( )
4  {
5      using namespace std::literals;
6      std::string name = "张三"s; //s 表明 std::string 类型，而不是 const char *型
7      auto during1 = 10ms; //10 毫秒
8      auto during2 = 20s; //20 秒
9  }

```

示例中的 s、ms 是标准库中预定义的文字量后缀。用户也可以通过重载文字量运算符，定义自己的文字量及后缀，但有以下要求：

- (1) 对于参数，若为整数类型，必须使用 `unsigned long long int`；若为浮点类型，必须使用 `long double`；若为字符类型，必须是 `char`、`wchar_t`、`char8_t`、`char16_t` 或 `char32_t`；若为字符串，参数表必须采用以下形式之一：`(const char *, [size_t])`、`(const wchar_t *, [size_t])`、`(const char16_t *, [size_t])` 或 `(const char32_t *, [size_t])` 等。
- (2) 重载函数的返回值类型不做限制。
- (3) 后缀名可以由多个字符组成，区分大小写。由于可能存在编码问题，不建议使用汉字作为后缀名。
- (4) 后缀名必须以单个下划线 `_` 开头，并紧接在 `""` 之后。非 `_` 开头的后缀名保留给标准库使用。如果 `_` 不紧跟在 `""` 之后，可能会导致早期代码出现兼容性问题。

例如：

```

1  #include <iostream>
2  using namespace std;
3  //浮点型参数类型必为 long double
4  long double operator ""_w( long double val ) {
5      return val * 10000;

```

```

6  }
7  //整型参数类型必为 unsigned long long int
8  unsigned long long operator ""_w( unsigned long long val ) {
9      return val * 10000;
10 }
11 struct Student {
12     Student(const string & name, bool isMale )
13         : mName(name), mIsMale(isMale){ }
14     string mName;
15     bool   mIsMale;
16 };
17 //字符串型的参数表 (const char * sz, size_t len)
18 Student operator ""_Male( const char * sz, size_t len ) {
19     return Student( string { sz, len }, true );
20 }
21 //文字量后缀必为`_`开头，可多个字符
22 Student operator ""_Female( const char * sz, size_t len ) {
23     return Student( string { sz, len }, false );
24 }
25 int main( ) {
26     long double data1 = 1.25_w;          //1.25 万
27     unsigned long long data2 = 87_w;     //87 万
28
29     cout << fixed << data1 << endl;      //输出 12500.000000
30     cout << data2 << endl;                //输出 870000
31
32     Student s1 = "张三"_Male;
33     Student s2 = "李四"_Female;
34
35     //输出 张三    男
36     cout << s1.mName << "\t" << (s1.mIsMale ? "男" : "女") << endl;
37     //输出 李四    女
38     cout << s2.mName << "\t" << (s2.mIsMale ? "男" : "女") << endl;
39 }

```

10.5 重载的方针

当重载类型 T 的运算符时，应根据 C++ 标准的要求选择非成员形式或成员形式，具体如表 10-3 所示：

表 10-3 运算符的可重载形式

类别	运算符	可重载	成员形式	非成员形式
赋值运算符	=	是	可	禁止
赋值运算符	+=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=	是	可	可
算数运算符	(正号)+, (负号)-, (加法)+, (减法)-, (乘法)*, (除法)/, (模)%, (按位取反)~, (按位与)&, (按位或) , (按位异或)^, (位左移)<<, (位右移动)>>	是	可	可
递增递减运算符	(前置递增)++, (后置递增)++, (前置递减)--, (后置递减)--	是	可	可
逻辑运算符	(逻辑非)!, (逻辑与)&&, (逻辑或)	是	可	可
比较运算符	(等于)==, (不等于)!=, (小于)<, (大于)>, (小于等于)<=, (大于等于)>=, (三路比较, C++20 增)<=>	是	可	可
成员访问运算符	(取分量)[]	是	可	禁止
成员访问运算符	(间接引用/解引用)*, (取地址)&	是	可	可
成员访问运算符	(对象的成员)., (对象成员的指针).*	否	禁止	禁止
成员访问运算符	(指针的成员)->	是	可	禁止
成员访问运算符	(指针成员的指针)->*	是	可	可
函数调用运算符	()	是	可	禁止
逗号运算符	,	是	可	可
条件运算符	?:	否	禁止	禁止

在重载运算符时，如果可以选择成员函数或非成员函数形式，通常应优先选择成员函数形式。

具体指导原则如下：

1. 当二元运算符的左操作数必定是类型 T 的对象时，建议以成员函数的形式重载该运算符。例如，对于复合赋值运算符(如+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=)，其左操作数总是自定义类型 T 的对象，因此应当采用成员函数形式进行重载。特别地，C++标准规定，赋值运算符=和分量运算符[]必须以成员函数的形式重载。
2. 如果二元运算符的左操作数可能是类型 T 的对象，也可能不是，则建议以非成员函数的形式重载该运算符。例如，对于加法运算符+，表达式可以是 t + 1，也可能是 1 + t。只有以非成员函数形式重载才能同时适应这两种情况。同样地，对于减法-、乘法*、除法/、取模%、按位与&、按位或|、逻辑与&&、逻辑或||等运算符，也应采用非成员函数形式。

3. 当二元运算符的左操作数一定不是类型 T 的对象时，只能以非成员函数形式重载。例如，流运算符<<的左操作数是流对象，而非类型 T 的对象，因此只能使用非成员函数形式进行重载。
4. 对于一元运算符，由于其操作数必定是类型 T 的对象，建议采用成员函数形式重载。特别地，C++标准规定，成员访问->运算符必须以成员函数形式重载。
5. 无论采用哪种形式，重载运算符的语义都应与内置类型的相应运算符保持一致，并且不应改变运算符的常规使用习惯。例如，正确实现赋值运算符 operator=的方式应该是返回当前对象的引用，以便支持链式赋值，如 t1 = t2 = t3;。
6. 不要重载逻辑与(&&)、逻辑或(||)以及逗号(,)运算符。逻辑与和逻辑或具有短路求值机制，在遇到第一个决定性结果时会停止计算后续操作数，而逗号运算符则确保先计算左操作数再计算右操作数。一旦重载这些运算符，操作数作为函数参数传递，其计算顺序是不定的，从而无法保证原有的短路行为和计算结果的正确性。

10.6 例子

10.6.1 日期类

现定义一个拥有年月日数据的 Date 类，并通过重载运算符提供相关的访问接口，包括定向输出、取得前后日期，昨天和明天的日期，日期相差的天数、日期是星期几等，同时实现其它必要的辅助函数。具体如下：

```

1  //头文件
2  #pragma once
3  #include <iostream>
4  #include <string>
5
6  class Date; //前置声明
7  std::ostream & operator<< ( std::ostream & out, const Date & date ); //流输出
8
9  class Date {
10 public:
11     Date( ) : Date(1900,1,1) { } //委托构造
12     Date( int year, int month, int day ); //构造函数
13     Date & operator++( ); //后移到明天
14     Date & operator--( ); //前移到昨天
15     Date operator++( int ); //明天日期
16     Date operator--( int ); //昨天日期
17     Date & operator+=( int days ); //后移 days 天

```

```

18     Date & operator-=( int days );//前移 days 天
19     int operator-( const Date & other ) const; //两个日期的天数差
20 public: //getter
21     int year( ) const      { return mYear; }
22     int month( ) const     { return mMonth; }
23     int day( ) const       { return mDay; }
24     const std::string & weekDay( ) const; //星期几字符串
25 private:
26     bool checkValid( ) const;           //不变性验证
27     int  totalDaysFrom19000101( ) const; //自 19000101 到当前日期所经过的天数
28 private:
29     static bool isLeap( int year );      //是否是闰年
30 private:
31     static const int daysInMonth[ 13 ]; // { 0,31,28,31,30,31,30,31,31,30,31,30,31 };
32     static const std::string weekDayNames[ 7 ]; //"星期日"到"星期六"的字符串
33 private:
34     int mYear;
35     int mMonth;
36     int mDay;
37 };

```

```

1  //实现文件
2  #include <cassert>
3  #include <string>
4  #include <iomanip>
5  #include "Date.h"
6
7  //静态数据初始化
8  const int Date::daysInMonth[ 13 ] { 0,31,28,31,30,31,30,31,31,30,31,30,31 };
9  const std::string Date::weekDayNames[ 7 ]
10     { "星期日","星期一","星期二","星期三","星期四","星期五","星期六" };
11
12 //构造函数
13 Date::Date( int year, int month, int day )
14     :mYear( year ), mMonth( month ), mDay( day )
15 {
16     if (!checkValid( )) {
17         std::cerr << "无效日期" << std::endl;
18         assert( false );
19     }
20 }
21 //不变性验证
22 bool Date::checkValid( ) const {

```

```

22     bool yearValid = ( mYear >= 1900 && mYear <= 2500 );
23     bool monthValid = ( mMonth > 0 && mMonth <= 12 );
24     bool dayValid = ( mDay > 0 && mDay <= 31 );
25     if (!yearValid || !monthValid || !dayValid) return false;
26
27     int add = ( ( mMonth == 2 && Date::isLeap( mYear ) ) ? 1 : 0 );
28     return mDay <= daysInMonth[ mMonth ] + add;    //29 日, 30 日, 31 日是否合法
29 }
30 //自 19000101 到当前日期所经过的天数
31 int Date::totalDaysFrom19000101( ) const {
32     int result = 0;
33     for (int y = 1900; y < mYear; ++y)
34         result += 365 + Date::isLeap( y );
35     for (int m = 1; m < mMonth; ++m) {
36         result += daysInMonth[ m ];
37         if (m == 2 && isLeap( mYear ))
38             ++result;
39     }
40     result += mDay - 1;
41     return result;
42 }
43 //类方法: 是否闰年
44 bool Date::isLeap( int year ) {
45     assert( year >= 1900 && year < 2200 );
46     return ( year % 400 == 0 || ( year % 4 == 0 && year % 100 != 0 ) );
47 }
48 //后移到明天
49 Date & Date::operator++( ) {
50     int add = ( mMonth == 2 && Date::isLeap( mYear ) ) ? 1 : 0;
51     ++mDay;
52     if (mDay > daysInMonth[ mMonth ] + add) {
53         ++mMonth;
54         mDay = 1;
55         if (mMonth > 12) {
56             ++mYear;
57             mMonth = 1;
58         }
59     }
60     return *this;
61 }
62 //前移到昨天
63 Date & Date::operator--( ) {
64     if (--mDay <= 0) {
65         --mMonth;

```

```

66         int add = ( mMonth == 2 && Date::isLeap( mYear ) ) ? 1 : 0;
67         mDay = daysInMonth[ mMonth ] + add;
68         if (mMonth <= 0) {
69             mMonth = 12;
70             --mYear;
71         }
72     }
73     return *this;
74 }
75 //明天日期
76 Date Date::operator++( int ) {
77     Date result( *this );
78     ++( *this );
79     return result;
80 }
81 //昨天日期
82 Date Date::operator--( int ) {
83     Date result( *this );
84     --( *this );
85     return result;
86 }
87
88 //后移 days 天
89 Date & Date::operator+=( int days ) {
90     for (int i = 1; i <= days; ++i) ++( *this );
91     return *this;
92 }
93 //前移 days 天
94 Date & Date::operator-=( int days ) {
95     for (int i = 1; i <= days; ++i) --( *this );
96     return *this;
97 }
98 //两个日期的天数差
99 int Date::operator-( const Date & other ) const {
100     int days1 = totalDaysFrom19000101( );
101     int days2 = other.totalDaysFrom19000101( );
102     return days1 - days2;
103 }
104 //星期几字符串
105 const std::string & Date::weekDay( ) const {
106     int days = totalDaysFrom19000101( ); //1900 年 1 月 1 日为星期一
107     return weekDayNames[ ( days + 1 ) % 7 ];
108 }
109 //输出 Date 到流

```

```

110 std::ostream & operator<<( std::ostream & out, const Date & date ) {
111     out << date.year( ) << "年";
112     out << std::setw( 2 ) << std::setfill( '0' ) << date.month( ) << "月";
113     out << std::setw( 2 ) << std::setfill( '0' ) << date.day( ) << "日";
114     out << "\t" << date.weekDay( );
115     return out;
116 }

```

```

1  // 测试用主函数
2  #include <iostream>
3  #include <iomanip>
4  using namespace std;
5  #include "Date.h"
6  int main( )
7  {
8      Date d0;
9      Date d1( 1949, 10, 1 );
10     Date d2( 2008, 8, 8 );
11     Date d3( 2025, 9, 3 );
12
13     cout << d0 << endl;           //1900 年 01 月 01 日 星期一
14     cout << d1 << endl;           //1949 年 10 月 01 日 星期六
15     cout << ++d1 << endl;         //1949 年 10 月 02 日 星期日
16     cout << ( d1 += 2 ) << endl;   //1949 年 10 月 04 日 星期二
17     cout << d2-- << endl;         //2008 年 08 月 08 日 星期五
18     cout << d2 << endl;           //2008 年 08 月 07 日 星期四
19     cout << d3 << endl;           //2025 年 09 月 03 日 星期三
20     cout << "天数差: " << d3 - Date( 2000, 1, 1 ) << endl; //天数差: 9377
21 }

```

10.6.2 封装函数

函数参数表中的参数可以是多种类型，除了常见类型之外，参数也可以是一个函数。例如：

```

1  #include <iostream>
2  int f1( int n ) { return n + 1; }
3  int f2( int n ) { return n * 2; }
4  using FType = int (*)(int ); //定义形如 int xyz(int) 的函数的类型为 FType
5
6  int func( int m, int added, FType f ) { return f(m) + added; }
7  int main( ) {
8      std::cout << func( 3, 2, f1 ) << std::endl; //输出 6
9      std::cout << func( 4, 5, f2 ) << std::endl; //输出 13
10 }

```

在 C++ 中，函数参数更常见地是采用传递对象而不是传递函数。首先，传递对象是最常用的传参形式，相比给函数传递函数参数，用户使用传递对象更自然也更熟悉；其次，对象本身可以有自己的数据成员，所以对象含有的传递信息可以远多于传递函数。下面的代码演示了通过封装函数并让对象存放更多数据供函数使用：

```

1  #include <iostream>
2  int f1( int n ) { return n + 1; }
3  int f2( int n ) { return n * 2; }
4  using FType = int (*) (int ); //定义形如 int xyz(int) 的函数的类型为 FType
5
6  class Func {
7  public:
8      Func( int val, FType f ) : mValue(val), mF( f ) { }
9      void setVal( int val ) { mValue = val;}
10     int operator( )(int m, int added ) const //通过重载运算符实现
11         { return mF(m)+ added + mValue; }
12 private:
13     int mValue;
14     FType mF;
15 };
16 int main( ) {
17     Func myF(10,f1);
18     std::cout << myF(3,2) << std::endl;    //输出 16
19     myF.setVal(15);
20     std::cout << myF(3,2) << std::endl;    //输出 21
21
22     Func yourF(20,f2);
23     std::cout << yourF(4,5) << std::endl; //输出 33
24     yourF.setVal(100);
25     std::cout << yourF (3,2) << std::endl; //输出 113
26 }

```

这里定义的 Func 类，通过重载运算符(), 可以将全局函数封装为对象，从而将 Func 类型的对象当作全局函数使用，同时还可以扩展函数的语义。在 C++ 中，被封装的函数除了可以是全局函数之外，还可以是类的实例方法、类方法或者 Lambda 表达式等，因此，封装函数更通用的方法是使用 std::function 类模板并结合 std::bind 函数。下面的代码是一个应用示例：

```

1  #include <iostream>
2  #include <functional>
3
4  int f( int a, int b ) { return a + b; }

```

```

5  class Foo {
6  public:
7      Foo( int num ) :mNum( num ) {}
8      int g( int a, int b, const int& c, int d ) {
9          return a + b + c + d + mNum;
10     }
11     static int k( int a, int b, int& c ) {
12         return a * 2 + b + c;
13     }
14 private:
15     int mNum;
16 };
17
18 int main( ) {
19     //封装全局函数
20     auto f1 = std::function<int( int, int )>( &f );
21     std::cout << f1( 2, 3 ) << std::endl;      //输出 5
22
23     //封装实例方法
24     using namespace std::placeholders;      //可以使用 _1,_2,_3,...
25     Foo foo { 6 };
26     int n = 10;
27     //实例方法 g 隐式传递的 this 指针, 值为&foo;
28     //第一个参数绑定第一个实参;第二个参数绑定第二个实参;
29     //第三个参数绑定为 n 的常引用;第四个参数绑定为 n 的值
30     auto f2 = std::bind( &Foo::g, &foo, _1, _2, std::cref( n ), n );
31     n = 8;
32     std::cout << f2( 2, 3 ) << std::endl;      //输出 29,等价 foo.g(2,3,n,10)
33
34     //封装类方法
35     n = 7;
36     //类方法 k 没有 this 指针;
37     //第一个参数绑定第二个实参;第二个参数绑定第一个实参;
38     //第三个参数绑定为 n 的引用;
39     auto f3 = std::bind( &Foo::k, _2, _1, std::ref( n ) );
40     std::cout << f3( 2, 6 ) << std::endl;      //输出 21,等价 Foo::k(6,2,n)
41
42     //lambda 表达式
43     auto f4 = []( int a, int& b, int c ) { return a + b * 2 + c * 3; };
44     std::cout << f4( 2, n, 4 ) << std::endl;    //输出 28, 2+7*2+4*3=28
45
46     //封装 lambda 表达式
47     //lambda 表达式的第一个参数绑定第一个实参,
48     //第二个参数绑定为 n 的引用, 第三个参数绑定为数值 10

```

```

49     auto f5 = std::bind( f4, _1, std::ref( n ), 10 );
50     std::cout << f5( 3 ) << std::endl;           //输出 47, 3+7*2+10*3=47
51 }

```

从上述例子中可以看出, 通过使用 `std::bind` 能够将任意函数封装为 `std::function` 对象, 由于 `std::bind` 可以固定部分参数、改变实参顺序等, 使得当使用 `std::function` 对象作为函数参数时, 可以传递的函数更灵活。例如:

```

1  #include <iostream>
2  #include <functional>
3
4  int f( int a, int b ) { return a * b; }
5  class Foo {
6  public:
7      int test( std::function<int( int, int )> f, int param1, int param2 )
8          { return f( param1, param2 ); }
9      int g( int a, int b ) { return a + b * 2; }
10     static int s( int a, int b, int c ) { return a + b * 2 + c * 3; }
11 };
12
13 int main( ) {
14     using namespace std::placeholders;
15     Foo foo;
16     //输出 12, 等价于调用 f(3,4)
17     std::cout << foo.test( f, 3, 4 ) << std::endl;
18     //输出 16, 改变了参数顺序, 等价于调用 foo.g(6,5)=6+5*2=16
19     std::cout << foo.test( std::bind( &Foo::g, &foo, _2, _1 ), 5, 6 ) << std::endl;
20     //输出 43, 等价于调用 Foo::s(7,6,8)=7+6*2+8*3=43
21     std::cout << foo.test( std::bind( &Foo::s, _1, 6, _2 ), 7, 8 ) << std::endl;
22     //输出 55
23     std::cout<<foo.test([](int a,int b) { return a * 5 + b;}, 9, 10 ) << std::endl;
24 }

```

例子中可以给 `test` 成员函数传递普通函数、实例方法、类方法和 `lambda` 表达式, 结合 `std::bind` 的使用, 甚至可以传递原参数表不匹配的函数。